

- Application flow
 - Purpose of this document
 - One-line flow
 - User-facing dashboard flow
 - Step 1. Client input
 - Step 2. Run analysis
 - Step 3. Results unlock
 - Step 4. Show results
 - Internal application pipeline
 - Stage 1. Video input
 - Stage 2. Frame extraction
 - Stage 3. Pose estimation
 - Stage 4. Skeleton overlay video
 - Stage 5. Landmark validation
 - Stage 6. Frame-level biomechanics
 - Stage 7. Rep segmentation
 - Stage 8. Per-rep feature extraction
 - Stage 9. Fatigue, risk, and mistake labels
 - Stage 10. RL state construction
 - Stage 11. Policy action selection
 - Stage 12. Reward calculation
 - Stage 13. Coaching output generation
 - Stage 14. User-side explanation
 - Stage 15. Technical-side explanation
 - Stage 16. Visual evidence
 - Stage 17. Evaluation and artifacts
 - Current real-video result
 - Failure and edge-case handling
 - End-to-end file map
 - Panel-ready explanation

Application flow

Purpose of this document

This document explains end-to-end flow of Adaptive RL Gym Coach from user input to final coaching output. It is written for review-panel questions around pipeline design, ML inference, RL decision-making, explainability, data contracts, and demo behavior.

One-line flow

Client details + squat video -> pose landmarks -> biomechanics features -> fatigue/risk/mistake detection -> RL state -> coaching action -> safety filter -> user and technical explanation -> submission artifacts

User-facing dashboard flow

Step 1. Client input

File:

app.py

User enters:

Input	Purpose
Name	Creates local user profile ID
Skill level	Encoded into RL state as beginner, intermediate, or advanced
Height in cm	Validated profile field
Injury notes	Stored locally in profile
Video upload or bundled sample	Source for pose analysis

Profile generation:

name -> slug user_id -> data/profiles/{user_id}.json

Default baseline angles:

knee_min: 136
hip_min: 148
torso_lean_max: 24

Step 2. Run analysis

User clicks:

Run analysis

Button is enabled only when:

name exists AND uploaded video exists OR bundled sample is selected

System runs:

```
_create_profile()  
_uploaded_video_to_temp() or sample video path  
_run_video_analysis()
```

Step 3. Results unlock

After analysis finishes:

```
analysis_complete = True  
results_visible = False
```

Dashboard reruns so:

Show results

becomes enabled.

Step 4. Show results

User clicks:

Show results

Dashboard switches to results view with:

- Session summary cards.
- User side explanation.
- Technical side explanation.
- Fatigue and injury-risk chart.
- Rep table.
- Pose overlay video.
- Evidence frames.

Internal application pipeline

Stage 1. Video input

Input can be:

```
Uploaded video  
data/sample_videos/real_squat_sample.mov
```

Video extensions accepted:

```
mp4, mov, avi, mkv
```

Main function:

```
analyze_video(video_path, output_dir, profile)
```

Stage 2. Frame extraction

File:

```
src/pose.py
```

OpenCV opens video:

```
cv2.VideoCapture(video_path)
```

For every frame:

```
BGR frame -> RGB frame -> MediaPipe Pose inference
```

Why RGB conversion matters:

- OpenCV reads BGR.
- MediaPipe expects RGB.
- Wrong color order can reduce pose detection quality.

Stage 3. Pose estimation

Model:

```
MediaPipe Pose
```

For each frame, system stores selected landmarks:

```
left_shoulder  
right_shoulder  
left_hip  
right_hip  
left_knee  
right_knee  
left_ankle  
right_ankle
```

Output data contract:

```
outputs/latest_session/landmarks.csv
```

Columns:

```
frame,timestamp,landmark_name,x,y,z,visibility
```

Review-panel explanation:

- This is deep learning inference stage.
- Model converts raw pixels into structured body keypoints.
- Downstream logic uses keypoints, not raw video pixels.

Stage 4. Skeleton overlay video

MediaPipe landmarks are drawn on each frame.

Output:

```
outputs/latest_session/pose_overlay.mp4
```

Video compatibility:

```
OpenCV mp4v -> ffmpeg H.264 conversion when ffmpeg is available
```

Why:

- Browser playback often requires H.264 MP4.
- Final overlay verified as h264/avc1/yuv420p .

Stage 5. Landmark validation

For CSV analysis, system validates required columns:

```
frame  
timestamp  
landmark_name  
x  
y
```

```
z
visibility
```

If required columns are missing:

```
ValueError("Landmark CSV missing columns: ...")
```

Why:

- Prevents silent wrong analysis.
- Makes data contract explicit.

Stage 6. Frame-level biomechanics

File:

```
src/biomechanics.py
```

First, landmarks are grouped by frame. Low-confidence landmarks are filtered:

```
visibility >= 0.3
```

Each valid frame produces:

Metric	How calculated	Meaning
Knee angle	hip-knee-ankle angle, averaged left and right	Squat depth and knee flexion
Hip angle	shoulder-hip-knee angle, averaged left and right	Hip hinge and depth
Torso lean	mid-hip to mid-shoulder angle	Forward lean
Knee tracking proxy	knee-to-ankle horizontal offset divided by hip width	Knee path alignment
Knee asymmetry	left-right knee angle difference divided by 180	Uneven movement proxy

Stage 7. Rep segmentation

Input:

```
knee angle series over time
```

Algorithm:

```
standing -> knee angle drops below low threshold -> bottom -> knee angle rises above high threshold -> rep complete
```

Thresholds:

```
low = min_angle + 0.38 * angle_range
high = min_angle + 0.72 * angle_range
minimum_rep_frames = max(8, fps * 0.45)
```

Why knee angle:

- Squat descent and ascent show strong knee-angle phase change.
- It gives simple, interpretable rep boundaries.

If no complete cycle is detected:

```
System creates one fallback rep from first frame to last frame with lowest knee angle as bottom.
```

Why fallback exists:

- Avoids crashing on short synthetic or partial sessions.
- Still lets pipeline produce interpretable output.

Stage 8. Per-rep feature extraction

Output:

```
outputs/latest_session/rep_features.csv
```

Each rep stores:

Output	Use
knee_angle_min	Depth and shallow squat detection
hip_angle_min	Hip movement signal
torso_lean_max	Forward lean detection
tempo_sec	Fatigue signal
smoothness	Stability signal
fatigue_score	Coaching and safety signal
injury_risk	Coaching and safety signal
mistake_label	Main interpretable issue
knee_tracking_proxy	Knee path issue
depth_proxy	Movement quality and reward
evidence_frame	Frame shown in UI

Stage 9. Fatigue, risk, and mistake labels

Fatigue uses:

```
tempo slowdown  
range of motion loss  
movement jitter
```

Injury risk uses:

```
torso lean  
knee tracking  
left-right asymmetry
```

Mistake labels:

```
knee_tracking  
forward_lean  
shallow_squat  
unstable_motion  
none
```

Review-panel explanation:

- These are not black-box labels.
- Each label comes from a visible metric and threshold.
- This supports explainable AI.

Stage 10. RL state construction

File:

```
src/coaching.py
```

For each rep, state vector is built:

```
[  
    knee_angle_min / 180,  
    hip_angle_min / 180,  
    torso_lean_max / 90,  
    rep_id / 20 capped at 1,  
    fatigue_score,  
    injury_risk,  
    repeated_mistakes / 5 capped at 1,  
    previous_action_id / 7,  
    skill_encoding  
]
```

Why normalized:

- Neural networks train better with small stable numeric ranges.

- Different raw features become comparable.
- DQN observation space is defined as values from 0 to 1.

Stage 11. Policy action selection

Policy loading:

```
load_policy(models/coach_policy)
```

If DQN zip exists:

```
models/coach_policy.zip -> StableBaselinesPolicy -> SafetyPolicy
```

If DQN cannot load:

```
HeuristicPolicy
```

Actions:

```
verbal_cue  
joint_highlight  
slow_tempo  
adjust_difficulty  
recommend_rest  
no_feedback  
demonstration  
breathing_cue
```

Review-panel explanation:

- DQN chooses discrete coaching action.
- Safety policy can override DQN when fatigue, risk, or mistake threshold is high.
- Final action is what user sees.

Stage 12. Reward calculation

Reward function considers:

Term	Effect
Next-rep improvement	Adds reward
Lower risk	Adds relative reward through improvement
Current risk	Subtracts reward
Current fatigue	Subtracts reward
Repeated mistake	Subtracts reward
Extra correction	Small penalty
Correct rest recommendation	Adds reward
Correct joint highlight	Adds reward
Correct no feedback	Adds reward

Why reward design matters:

- RL should not give feedback constantly.
- RL should prioritize safer movement.
- RL should handle fatigue and repeated mistakes over sequence, not one frame only.

Stage 13. Coaching output generation

Output:

```
outputs/latest_session/coach_outputs.json
```

Each rep output includes:

```
rep_id
state
action
reward
problem
reason
correction
evidence_frame
```

Example meaning:

- Problem: what went wrong.
- Reason: metric or threshold behind decision.
- Correction: coaching instruction.
- Evidence frame: frame shown to support decision.

Stage 14. User-side explanation

File:

```
src/dashboard.py
```

User side converts numeric scores to simple labels:

Score	Label
< 0.35	low
< 0.65	medium
>= 0.65	high

Form summary:

Bad rep ratio	Label
0 bad reps	good form
1 percent to 30 percent	mostly good with small corrections
31 percent to 60 percent	needs correction
More than 60 percent	poor form pattern

Purpose:

- Non-technical user sees trainer-style explanation.
- No ML jargon needed.
- Result focuses on what to fix next.

Stage 15. Technical-side explanation

Technical side shows:

- Pipeline summary.
- Profile baseline.
- Per-rep metric values.
- Triggered condition.
- Raw model action.
- Final action.
- Decision source.
- Decision reason.
- Reward.
- State vector.
- Evidence frame.

Purpose:

- Review panel can trace every conclusion from pose data to final advice.
- Shows ML and RL concepts clearly.
- Avoids black-box claims.

Stage 16. Visual evidence

Visual outputs:

```
outputs/latest_session/pose_overlay.mp4
outputs/latest_session/evidence_frames/evidence_frame_76.jpg
outputs/latest_session/evidence_frames/evidence_frame_178.jpg
outputs/latest_session/evidence_frames/evidence_frame_273.jpg
```

Why important:

- User can see skeleton overlay on original video.
- Evidence frame links technical metric to visible posture.
- Panel can verify analysis is based on actual video, not static text.

Stage 17. Evaluation and artifacts

Evaluation command:

```
python -m src.evaluate
```

Generated files:

```
outputs/evaluation_metrics.csv
outputs/reward_curve.svg
outputs/fatigue_over_reps.svg
outputs/injury_risk_over_reps.svg
```

Report command:

```
python report/build_report.py
```

Slide command:

```
node slides/build_deck.mjs
```

Submission package:

```
Adaptive_RL_Gym_Coach_Submission.zip
```

Current real-video result

Using bundled real sample video:

```
data/sample_videos/real_squat_sample.mov
```

Current output summary:

Metric	Value
Detected reps	17
Bad-form reps	17
Average fatigue	0.341
Average injury risk	0.659
Main issue	knee tracking
Joint highlight actions	15
Rest recommendations	2

Plain-language interpretation:

System detected 17 squats. Most reps showed knee tracking issue. Main correction is to keep knees aligned with toes and move with control. Rest was recommended on reps 14 and 16 because fatigue or risk increased.

Technical interpretation:

Knee tracking proxy crossed threshold on detected reps. This increased injury-risk proxy. State vector included knee angle, hip angle, torso lean, fatigue, risk, repeated mistake count, previous action, and skill encoding. DQN or fallback policy proposed action. Safety policy selected final joint highlight or rest recommendation.

Failure and edge-case handling

Case	Behavior
Missing video path	Raises clear video file error
Video cannot open	Raises clear video open error
Missing landmark CSV columns	Raises landmark column validation error
No full rep cycle	Creates fallback rep when landmarks exist
No reps after analysis	Dashboard shows clear error
DQN unavailable	Uses heuristic policy
ffmpeg unavailable	App still writes overlay, but browser playback may depend on codec

End-to-end file map

```
app.py
-> creates profile
-> calls analyze_video
-> loads policy
-> makes coaching outputs
-> renders dashboard

src/pose.py
-> video frames
-> MediaPipe Pose landmarks
-> overlay video
-> evidence frames

src/biomechanics.py
-> frame metrics
-> rep segmentation
-> rep features
-> fatigue/risk/mistake labels

src/coaching.py
-> state vector
-> DQN or heuristic action
-> safety policy
-> reward
-> explanation

src/dashboard.py
-> user summary
-> technical trace
-> score labels

src/rl_env.py
-> synthetic RL environment
-> state transitions
-> reward during training
```

Panel-ready explanation

The application first uses deep learning pose estimation to convert each squat video frame into body landmarks. Then it applies biomechanical feature extraction to calculate joint angles, torso lean, knee tracking, tempo, smoothness, fatigue, and injury-risk proxies. Each repetition becomes one RL state. A DQN policy selects coaching action, and a safety layer overrides risky choices when needed. Final output is explainable because dashboard shows both human-friendly correction and technical trace with metrics, thresholds, state vector, action, reward, and evidence frame.